4. Math and Number Theory

Contents

Math

```
Stirling numbers of the first kind : the number of permutations
    of n elements with k disjoint cycles
S(n,k) = n * S(n-1,k) + S(n-1,k-1)
Sum k = 0 -> n of S(n,k) = n!
```

```
Stirling numbers of the second kind : the number of ways to partition
a set of n elements into k nonempty subsets
S(n,k) = k * S(n-1,k) + S(n-1,k-1)
Sum k = 0 -> n of S(n,k) = Bn

Bell numbers : the possible partitions of a set into nonempty subsets
Sum k = 0 -> n-1 of n-1Ck * Bk = Bn

Sum of first n even numbers : n*(n+1)
Sum of first n odd numbers : n*n

Sum of squares of first n numbers:
n*(n+1)*(2*n+1)/6
Sum of squares of first n even numbers:
2*n*(n+1)*(2*n+1)/3
Sum of squares of first n odd numbers:
n*(2*n+1)*(2*n-1)/3

Number of ways to pick equal number of elements from two sets : (n+m)C(m)

Sum of phi(d) for all d | n is equal to n.
Number of pairs (x,y) that satisfy x+y=n and gcd(x,y)=1 is phi(n).

Sum(nCk) for k [0,n] = 2^n
Sum(mCk) for all m [0,n] = (n+1)C(k+1)
Sum((n+k)Ck) for all k [0,m] = (n+m+1)C(m)
Sum((nCk)^2) for all k [0,n] = (2*n)Cn
Sum(i*nCi) for all i [1,n] = n*2^(n-1)
Sum((n-i)Ci) for all i [0,n] = F(n+1)
Number of arrays with size n and sum m = (n-1+m)C(m) = (n-1+m)C(n-1)

P(A|B) is the probability of event A given that event B happened.
P(A&B) is the probability of events A and B happening.

P(A|B) = (P(B|A) * P(A) ) / P(B)
P(A|B) = P(A&B) / P(B)

Divisibility:
2 if the rightmost digit is divisible by 2
3 if the sum of the digits is divisible by 3
4 if the number formed by the last two digits is divisible by 4
5 if the rightmost digit is 0 or 5
6 if it is divisible by 2 and 3
7 if The number formed by all digits except the right-most digit -
    (2 * right-most digit) is divisible by 7
8 if the number formed by the last 3 digits is divisible by 8
9 if the sum of the digits is divisible by 9

-Getting half of the binomial expansion (Only odd indices or only even indices)
    by using (a+b)^n and (a-b)^n and adding both of them

To solve quadratic equation a*x^2 + b*x + c = 0
x = -b (+-)sqrt(b^2-4*a*c) / (2 * a)

-Sum of geometric series ar^i (i from 0 to n) = a(1 - r^n) / (1 - r)
-Sum of geometric series ar^i (i from 0 to infinity) = a / (1 - r)
```

```
- XOR from 1 to n = n%4 [n, 1, n+1, 0]

- n can be sum of 2 squares if n has no p^k [p = 3 % 4, k is odd]
    in its prime factorization
- n can be sum of 3 squares if n is not in form [4^a (8b + 7)]
- n always can be sum of >= 4 squares (remaining are zeros)

=============== FADY THE GOOOOOAAT ===============
- sum of divisors
 prime^power * prime2^power2 * ...

 ((prime^(power + 1) - 1) / (prime - 1)) * ((prime2^(power2 + 1) - 1) / (prime2 - 1)) * ...
 ================================================
 a % m == b
 a and m not coprime
 g = gcd(a, m)
 (a / g) % (m / g) = b / g

 a^x % m == b
 a and m not coprime
 g = gcd(a, m)
 (a^(x-1) * (a / g)) % (m / g) = b / g
 ================================================================
 a^(power%phi(m)) % m;
 ================================================================
 count balanced brackets
 r=n/2  ||  or r = number of opened brackets
 nCr(n, r) - nCr(n, r-1)
 ================================================================
 // different n*n grids whose each square have m colors
 // if possible to rotate one of them so that they look the same then they same
 t = n * n;
 total = (fp(m, t)
     + fp(m, (t + 1) / 2)
     + 2 * fp(m, (t / 4) + (n % 2))) % mod;
 total = mul(total, fp(4, mod - 2));
 ================================================================
 biggest divisors
 735134400 1344 => 2^6 3^3 5^2 7 11 13 17
 73513440 768
 ================================================================
 for (int x = mask; x > 0; x = (x - 1) & mask)
 get all x such that mask = mask | x
 ================================================================
 sum from 1 to n: i * nCr(n, i) = n * (1LL << (n - 1))
 sum of odd between [1, n]  = ((n+1) / 2)^2
 sum of even between [1, n] = (n/2) * (n/2 + 1)

ll sum_total(ll l, ll r) { // sum [l, r]
    return (r - l + 1) * (l + r) / 2;
}

ll sum_even(ll l, ll r) { // sum even [l, r]
    if (l % 2 != 0) ++l;
    if (r % 2 != 0) --r;
    if (l > r) return 0;
```

```
    ll n = (r - l) / 2 + 1;
    return n * (l + r) / 2;
}

ll sum_odd(ll l, ll r) { // sum odd [l, r]
    if (l % 2 == 0) ++l;
    if (r % 2 == 0) --r;
    if (l > r) return 0;
    ll n = (r - l) / 2 + 1;
    return n * (l + r) / 2;
}

ll arithm1(ll l, ll r, ll a, ll d) { // [l, r] starting a and diff d
    if (d == 0) return (a >= l && a <= r) ? a : 0;
    ll n1 = ((l - a + d - (d > 0 ? 1 : -1)) / d);
    ll first = a + n1 * d;
    if ((d > 0 && first > r)
        || (d < 0 && first < r))
            return 0;
    ll n2 = ((r - a) / d);
    ll last = a + n2 * d;
    ll n = (n2 - n1 + 1);
    return n * (first + last) / 2;
}

ll arithm2(ll a, ll d, ll n) { // starting a, diff d, n terms
    return n * (2 * a + (n - 1) * d) / 2;
}
 */

ll phi(ll x) { // sqrt(x)
    ll ans = x;
    for(ll i = 2; i * i <= x; i++) {
        if(x % i == 0) {
            while(x % i == 0) x /= i;
            ans -= ans / i;
        }
    }
    if(x > 1) ans -= ans / x;
    return ans;
}

array<ll, 2> CRT(ll a1, ll m1, ll a2, ll m2) {
    // x = a1 % m1, x = a2 % m2
    a1 %= m1, a2 %= m2;
    auto [g, q1, q2] = eGcd(m1, -m2);
    if ((a2 - a1) % g) return {-1, -1};
    ll lcm = m1 / g * m2;
    ll m = m2 / g;
    q1 = (a2 - a1) / g % m * q1 % m;
    ll res = (a1 + m1 * q1) % lcm;
    if (res < 0) res += lcm;
    return {res, lcm};
}

ll BSGS(ll a, ll b, ll p) { // a^x = b (mod p)
```

```
    a %= p, b %= p;
    if(b == 1) return 0;
    if(a == 0) return b == 0? 1: -1;
    int add = 0;
    ll g, tmp = 1;
    while ((g = gcd(a, p)) > 1) {
        if(b % g) return -1;
        p /= g, b /= g, tmp = tmp * (a / g) % p, ++add;
        if(tmp == b) return add;
    }
    b = b * modInv(tmp, p) % p;
    int n = (int)sqrtl(p) + 1;
    unordered_map<ll, int> mp;
    for (ll q = 0, cur = 1; q <= n; ++q)
        mp.emplace(cur, q), cur = cur * a % p;
    ll an = 1;
    for (ll i = 0; i < n; ++i) an = an * a % p;
    an = modInv(an, p);
    for (ll i = 0, cur = b; i <= n; ++i) {
        auto it = mp.find(cur);
        if(it != mp.end()) return i * n + it->second + add;
        cur = cur * an % p;
    }
    return -1;
}

int sumNPowerM(int n, int m) { // 1^m + 2^m ... n^m
    int k = m + 3;
    vector<int> res(k);
    for(int i = 1; i < k; i++) res[i] = (res[i - 1] + fp(i, m)) % mod;
    if(n < k) return res[n];
    int facK = k;
    vector<int> p(k); p[0] = 1;
    for(int i = 1; i < k; i++) {
        p[i] = int(p[i - 1] * 1LL * (n - i) % mod);
        facK = int(facK * 1LL * i % mod);
    }
    vector<int> inv(k + 1), s(k + 1);
    inv[k] = fp(facK, mod - 2), s[k] = 1;
    for(int i = k - 1; i >= 0; i--) {
        s[i] = int(s[i + 1] * 1LL * (n - i) % mod);
        inv[i] = int(inv[i + 1] * 1LL * (i + 1) % mod);
    }
    int ans = 0;
    for(int i = 1; i < k; i++) {
        int cur = int(res[i] * 1LL * p[i - 1] % mod * s[i + 1] % mod *
            inv[i - 1] % mod * inv[k - i - 1] % mod);
        if((k - i + 1) & 1) cur = (mod - cur) % mod;
        ans = (ans + cur) % mod;
    }
    return ans;
}
```

ModInt

```cpp
template <int M> struct ModInt {
    template <class T> T binpow(T a, ll b) {
        T res = 1;
        while (b) { if (b & 1) res *= a; a *= a, b >>= 1; }
        return res;
    }
    int v;
    ModInt() : v(0) {}
    ModInt(ll v_) {
        v = v_ % M;
        if (v < 0) v += M;
    }

    bool operator==(ModInt o) const { return v == o.v; };
    bool operator!=(ModInt o) const { return !(*this == o); };

    // ModInt add(ModInt a, ModInt b) { return ((a.v + b.v % M)+M)%M;}
    // ModInt sub(ModInt a, ModInt b) { return add(a, -b); }
    // ModInt mul(ModInt a, ModInt b) { return 1ll * a.v * b.v % M; }
    // ModInt div(ModInt a, ModInt b) { return mul(a, binpow(b, M-2)); }

    ModInt &operator+=(ModInt o) { v = (v + o.v) % M; return *this; }
    ModInt &operator-=(ModInt o) { v = (v - o.v + M) % M; return *this; }
    ModInt &operator*=(ModInt o) { v = 1ll * v * o.v % M; return *this; }
    ModInt &operator/=(ModInt o) { return (*this *= binpow(o, M - 2)); }

    friend ModInt operator+(ModInt a, ModInt b) { return a += b; }
    friend ModInt operator-(ModInt a, ModInt b) { return a -= b; }
    friend ModInt operator*(ModInt a, ModInt b) { return a *= b; }
    friend ModInt operator/(ModInt a, ModInt b) { return a /= b; }
    ModInt operator-() { return 0 - *this; }

    friend istream &operator>>(istream &is, ModInt &a) {
        ll x; is >> x;
        a = ModInt(x);
        return is;
    }
    friend ostream &operator<<(ostream &os, ModInt a) { return os << a.v; }
    friend string to_string(ModInt a) { return to_string(a.v); }
};
using mint = ModInt<>;
```

Sieve / PHI up to n / 2D gcd()

```cpp
const int NS = 1e7;
const int NP = (NS/log(NS)) * (1 + 1.28 / log(NS));
int prSz;
int spf[NS], prm[NP];

auto pre_Sieve = []() {
    for (int i = 2; i < NS; i++){
        if(!spf[i]) spf[i] = prm[prSz++] = i;
        for(int j = 0; i * prm[j] < NS; j++) {
            spf[i * prm[j]] = prm[j];
            if(spf[i] == prm[j]) break;
```

```
        }
    }
    return 0;
}();

auto factors(int n) {
    vector<array<int, 2>> res;
    if(n < 2) return res;
    int p = spf[n];
    while(p > 1) {
        res.push_back({p, 0});
        while(n % p == 0) n /= p, res.back()[1]++;
        p = spf[n];
    }
    return res;
}

auto getDivisors(int _n) {
    auto _fac = factors(_n);
    int cnt = 1;
    for(auto [pr, pw] : _fac) cnt *= pw + 1;
    vector<int> res(1, 1); res.reserve(cnt);

    for(auto [pr, pw] : _fac)
        for(int i = int(res.size()) - 1; i >= 0; i--)
            for(int b = pr, j = 0; j < pw; j++, b *= pr)
                res.push_back(res[i] * b);
    sort(res.begin(), res.end());
    return res;
}

bool isPrime(ll n) {
    if(n < 4) return n > 1;
    if(n % 2 == 0 || n % 3 == 0) return false;
    for (ll i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    return true;
}

void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}

// 2d gcd in n^2
vector<vector<int>> GCD(N, vector<int>(N, 1));
for(int d = 2; d < N; ++d)
```

```
    for(int i = d; i < N; i += d)
        for(int j = d; j < N; j += d)
            GC[i][j] = d;
```

Sieve up to 1e9

```
vector<int> sieve(const int BN, const int Q = 17, const int L = 1 << 15) {
    using u = uint32_t; using u8 = uint8_t;
    static const u rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
    struct P {
        u p;
        u pos[8];
    };

    const u v = sqrt(BN), vv = sqrt(v);
    vector<bool> isp(v + 1, true);
    for (u i = 2; i <= vv; ++i)
        if (isp[i])
            for (u j = i * i; j <= v; j += i) isp[j] = false;

    const u rsize = BN > 60184 ? BN / (log(BN) - 1.1) :
        max(1.0, BN / (log(BN) - 1.11)) + 1 + 30;
    vector<int> primes = {2, 3, 5}; u psize = 3;
    primes.resize(rsize); vector<P> sprimes;

    u pbeg = 0, prod = 1;
    for (u p = 7; p <= v; ++p) {
        if (!isp[p]) continue;
        if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
        P pp = {p, {}};
        for (int t = 0; t < 8; ++t) {
            u j = p <= Q ? p : p * p;
            while (j % 30 != rs[t]) j += p << 1;
            pp.pos[t] = j / 30;
        }
        sprimes.push_back(pp);
    }

    vector<u8> pre(prod, 0xFF);
    for (size_t pi = 0; pi < pbeg; ++pi) {
        auto &pp = sprimes[pi];
        const u p = pp.p;
        for (int t = 0; t < 8; ++t) {
            const u8 m = ~(1 << t);
            for (u i = pp.pos[t]; i < prod; i += p) pre[i] &= m;
        }
    }

    const u block_size = (L + prod - 1) / prod * prod;
    vector<u8> block(block_size);
    u8* __restrict pblock = block.data();
    const u M = (BN + 29) / 30;

    for (u beg = 0; beg < M; beg += block_size, pblock -= block_size) {
        u end = min(M, beg + block_size);
```

```
        for (u i = beg; i < end; i += prod)
            memcpy(pblock + i, pre.data(), min(prod, end - i));
        if (beg == 0) pblock[0] &= 0xFE;

        for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
            auto &pp = sprimes[pi];
            const u p = pp.p;
            #pragma GCC unroll 8
            for (int t = 0; t < 8; ++t) {
                u i = pp.pos[t];
                const u8 m = ~(1 << t);
                for (; i < end; i += p) pblock[i] &= m;
                pp.pos[t] = i;
            }
        }

        for (u i = beg; i < end; ++i)
            for (u m = pblock[i]; m > 0; m &= m - 1)
                primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
    }

    while (psize > 0 && primes[psize - 1] > BN) --psize;
    primes.resize(psize); return primes;
}
```

Egcd, Linear Diaphontine

```
array<ll, 3> eGcd(ll a, ll b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = eGcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

ax0 + by0 = g ==> ax + by = c
x0 *= c/g, y0 *= c/g

all solutions:
           x = x0 + k * (b/g),
           y = y0 - k * (a/g);

int ceildiv(int a, int b) {
    if ((a ^ b) >= 0) return (a + b - 1) / b;
    else return a / b;
}

int flordiv(int a, int b) {
    if ((a ^ b) >= 0) return a / b;
    else return (a - b + 1) / b;
}

non-negative solution:
        k >= ceildiv(-x*g,b)
        k <= flordiv(x*g, a)

```

```
bool havenonnegsol(int a, int b, int c, int& x, int& y) {
    int g = egcd(a, b, x, y);
    x *= c/g;
    y *= c/g;
    int l1 = ceildiv(-x*g, b);
    int l2 = flordiv(y*g, a);
    return l1 <= l2;
}

// MOD INV
ll modInv(ll a, ll m) {
    ll x = 1, x1 = 0, q, t, b = m;
    while(b) {
        q = a / b;
        a -= q * b, t = a, a = b, b = t;
        x -= q * x1, t = x, x = x1, x1 = t;
    }
    assert(a == 1);
    return (x + m) % m;
}
```

CRT

```
using T = __int128;
// ax + by = __gcd(a, b)
// returns __gcd(a, b)
T extended_euclid(T a, T b, T &x, T &y) {
  T xx = y = 0;
  T yy = x = 1;
  while (b) {
    T q = a / b;
    T t = b; b = a % b; a = t;
    t = xx; xx = x - q * xx; x = t;
    t = yy; yy = y - q * yy; y = t;
  }
  return a;
}
// finds x such that x % m1 = a1, x % m2 = a2. m1 and m2 may not be coprime
// here, x is unique modulo m = lcm(m1, m2). returns (x, m). on failure, m = -1.
pair<T, T> CRT(T a1, T m1, T a2, T m2) {
  T p, q;
  T g = extended_euclid(m1, m2, p, q);
  if (a1 % g != a2 % g) return make_pair(0, -1);
  T m = m1 / g * m2;
  p = (p % m + m) % m;
  q = (q % m + m) % m;
  return make_pair((p * a2 % m * (m1 / g) % m +
                    q * a1 % m * (m2 / g) % m) %  m, m);
}
```

MillerRabin

```
ll binpower(ll base, ll e, ll mod) {
    ll result = 1;
```

```
    base %= mod;
    while (e) {
        if (e & 1)
            result = (__uint128_t)result * base % mod;
        base = (__uint128_t)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(ll n, ll a, ll d, int s) {
    ll x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (__uint128_t)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};

bool MillerRabin(long long n) {
    if (n < 2)
        return false;

    vector<int> primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    for (auto a : primes) {
        if (n == a) return true;
        if (n % a == 0)
            return false;
    }

    int r = 0;
    long long d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }

    for (int a : primes) {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
            return false;
    }
    return true;
}
```

Number of Divisors up to 1e18

```
namespace countdivisors {
    // for one second: 5000 1e18, 100000 1e9, 200000 1e6
    using ull = unsigned long long;
    using u128 = __uint128_t;
    static mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
```

```
u128 mul128(ull a, ull b, ull m) { return (u128)a * b % m; }
ull mul_mod(ull a, ull b, ull m) { return (ull)mul128(a, b, m); }
ull pow_mod(ull a, ull d, ull m) {
    ull r = 1;
    while (d) {
        if (d & 1) r = mul_mod(r, a, m);
        a = mul_mod(a, a, m);
        d >>= 1;
    }
    return r;
}
bool isPrime(ull n) {
    if (n < 2) return false;
    for (ull p : vector<ull>({2ULL,3,5,7,11,13,17,19,23,29,31,37}))
        if (n % p == 0) return n == p;
    ull d = n - 1, s = 0;
    while (!(d & 1)) d >>= 1, ++s;
    for (ull a : vector<ull>({2ULL,325,9375,28178,450775,9780504,1795265022})) {
        if (a % n == 0) continue;
        ull x = pow_mod(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool comp = true;
        for (ull r = 1; r < s; ++r) {
            x = mul_mod(x, x, n);
            if (x == n - 1) { comp = false; break; }
        }
        if (comp) return false;
    }
    return true;
}
ull pollards_rho(ull n) {
    if (n % 2 == 0) return 2;
    while (true) {
        ull c = uniform_int_distribution<ull>(1, n - 1)(rnd);
        auto f = [&](ull x){ return (mul_mod(x, x, n) + c) % n; };
        ull x = rnd() % n, y = x, d = 1;
        while (d == 1) {
            x = f(x); y = f(f(y));
            d = gcd<ull>(x > y ? x - y : y - x, n);
        }
        if (d < n) return d;
    }
}
void factor(ull n, map<ull,int>& cnt) {
    if (n < 2) return;
    if (isPrime(n)) { cnt[n]++; return; }
    ull d = pollards_rho(n);
    factor(d, cnt);
    factor(n/d, cnt);
}
ull ans(ull n) {
    map<ull,int> cnt;
    factor(n, cnt);
    ull res = 1;
    for (auto [p, e] : cnt) res *= (e + 1);
    return res;
```

```
    }
}
```

n mod 1 + n mod 2 + n mod 3 + ……. + n mod m

```
// n and m tends to(1e13) => time complexity(sqrt(n))
void solve(int idx){
    int k=n/idx;
    int st=n%idx%mod;
    int l=n/(k+1);
    int len=(idx-l)%mod;
    ans=(ans + st*len%mod + k%mod*(len*(len-1)/2%mod)%mod)%mod;
    if(l) solve(l);
}
```

n-th Fib Number

```
int fast_Fibonacci(int n) {
    int i = 1, h = 1, j = 0, k = 0, t;
    while (n > 0) {
        if (n % 2 == 1)
            t = j * h, j = i * h + j * k + t, i = i * k + t;
        t = h * h, h = 2 * k * h + t, k = k * k + t, n = n / 2;
    }
    return j;
}
```

Long Division

```
int a = 23, b = 5, n = 10;
vector<int> s;
for (int i = 0; i < n; i++) {
    unsigned long long k = a / b; // Integer division: gets the next digit
    a -= b * k;                   // Remove the part already represented
    a *= 10;                      // Shift remainder left (for next digit)
    s.push_back(k);              // Store the digit
}
```

Floor Values

```
// code to get all different values of floor(n/i)
for (ll l = 1, r = 1; (n/l); l = r + 1) { // O(sqrt)
    r = (n/(n/l));
    // q = (n/l), process the range [l, r]
}
```

Combinatorics

```
namespace combinatorics {
    vector<mint> fac_(1, 1), inv_(1, 1);
    void build(int N) {
        N += 10;
        fac_.assign(N, 1);
        inv_.assign(N, 1);
        for (int i = 2; i < N; i++)
            fac_[i] = fac_[i-1] * i;
        inv_[N-1] = 1/fac_[N-1];
        for (int i = N-2; i > 1; i--)
            inv_[i] = inv_[i+1] * (i+1);
    }
    inline mint nCr(int n, int r) {
        if(n < 0 || r < 0 || r > n) return 0;
        return fac_[n] * inv_[r] * inv_[n - r];
    }

    inline mint nCr(int64_t n, int64_t r) {
        if(n < 0 || r < 0 || r > n) return 0;
        r = min<int64_t>(r, n - r);
        mint ans = inv_[r];
        for(int64_t i = n - r + 1; i <= n; i++) ans = ans * i;
        return ans;
    }
    inline mint nPr(int n, int r) {
        if(n < 0 || r < 0 || r > n) return 0;
        return fac_[n] * inv_[n - r];
    }
    inline mint stars_and_bars(int n, int r){
        return nCr(n + r - 1, r - 1);
    }
    inline mint catalan(int n) {
        if(n < 0) return 0;
        return fac_[2 * n] * inv_[n] * inv_[n + 1];
    }
}
// using namespace combinatorics;
auto pre = []() { combinatorics::build(); return 0; }();
```

nCr, nPr without precomputation

```
ll nCr(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    r = min(r, n-r);
    ll ret = 1;
    for (int i = 0; i < r; i++)
        ret = ret * (n-i) / (i+1);
    return ret;
}
ll nPr(int n, int r) {
    if (r < 0 || r > n) return 0;
    ll ret = 1;
    for (int i = 0; i < r; ++i) {
        ret *= (n - i);
    }
```

```
    return ret;
}
```

NCR table

```
void preprocess_nCr() {
    for (int n = 0; n < N; n++) {
        C[n][0] = C[n][n] = 1;
        for (int r = 1; r < n; r++) {
            C[n][r] = (C[n - 1][r - 1] + C[n - 1][r]) % MOD;
        }
    }
}
```

Catalan numbers

```
const int MOD = ....
const int MAX = ....
int catalan[MAX];
void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=n; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1])% MOD;
            if (catalan[i] >= MOD)
                catalan[i] -= MOD;
        }
    }
}

// 1- Number of correct bracket sequence consisting of n opening and n
closing brackets.

// 2- The number of rooted full binary trees with n+1 leaves (vertices
are not numbered).
// A rooted binary tree is full if every vertex has
either two children or no children.

// 3- The number of ways to completely parenthesize n+1 factors.

// 4- The number of triangulations of a convex polygon with n+2 sides
// (i.e. the number of partitions of polygon into disjoint triangles by
using the diagonals).

// 5- The number of ways to connect the 2n points on a circle to form n
disjoint chords.

// 6- The number of non-isomorphic full binary trees with n internal
nodes (i.e. nodes having at least one son).

// 7- The number of monotonic lattice paths from point (0,0) to point
(n,n) in a square lattice of size nxn,
// which do not pass above the main diagonal (i.e. connecting (0,0) to
```

```
(n,n)).

// 8- Number of permutations of length n that can be stack sorted
// (i.e. it can be shown that the rearrangement is stack sorted if and
only if there is no such index i<j<k, such that ak<ai<aj).

// 9- The number of non-crossing partitions of a set of n elements.

// 10- The number of ways to cover the ladder 1..n using n rectangles
```

Matrix Exponentiation

```
template <typename T>
struct Matrix {
    int n, m;
    vector<T> mat;

    Matrix(int n, int m) : n(n), m(m) {
        mat.assign(n * m, 0);
    }
    Matrix(int n) : n(n), m(n) {
        mat.assign(n * n, 0);
    }

    void identity() {
        assert(n == m);
        fill(mat.begin(), mat.end(), 0);
        for (int i = 0; i < n; i++) {
            mat[i * m + i] = 1;
        }
    }

    T* operator[](int r) { return &mat[r * m]; }
    const T* operator[](int r) const { return &mat[r * m]; }

    Matrix operator*(const Matrix& o) const {
        assert(m == o.n);
        Matrix res(n, o.m);

        for (int i = 0; i < n; i++) {
            for (int k = 0; k < m; k++) {
                T val = mat[i * m + k]; if (val == 0) continue;
                for (int j = 0; j < o.m; j++) {
                    res[i][j] = (res[i][j] + 1ll * val * o[k][j]) % mod;
                    // res[i][j] = res[i][j] + val * o[k][j];
                }
            }
        }
        return res;
    }

    Matrix operator+(const Matrix& o) const {
        assert(o.n == n && o.m == m);
        Matrix ret(n, m);
        for (int i = 0; i < n; i++)
```

```
            for (int j = 0; j < m; j++) {
                ret[i][j] = ((*this)[i][j] + o[i][j]) % mod;
                // ret[i][j] = (*this)[i][j] + o[i][j];
            }
        return ret;
    }

    Matrix pow(ll k) const {
        assert(n == m);
        Matrix res(n), base = *this;
        res.identity();
        while (k) {
            if (k & 1) res = res * base;
            base = base * base;
            k >>= 1;
        }
        return res;
    }
};
```

K-th permutation

```
ll fac[21];
void preFac() {
    fac[0] = 1;
    for(int i = 1; i <= 20; ++i)
        fac[i] = fac[i - 1] * i;
}

vector<int> kth_permutation(int n, ll k) { // n^2
    vector<int> a(n), ans;
    iota(a.begin(), a.end(), 1);
    for (int i = n; i >= 1; i--) {
        ll f = fac[i - 1];
        ans.push_back(a[k / f]);
        a.erase(a.begin() + k / f);
        k %= f;
    }
    return ans;
}
```

Permutation Index

```
ll permutation_index(vector<int>& p) { // n^2
    int n = int(p.size());
    vector<int> a(n);
    iota(a.begin(), a.end(), 1);

    ll k = 0;
    for (int i = 0; i < n; ++i) {
        int j = int(find(a.begin(), a.end(), p[i]) - a.begin());
        k += j * fac[n - 1 - i];
        a.erase(a.begin() + j);
    }
```

```
    return k;
}
```

Berlekamp Massey

```
const ll MOD = 1e9+7;
ll add(ll a, ll b) { return (a + b) % MOD; }
ll sub(ll a, ll b) { return ((a - b) % MOD + MOD) % MOD; }
ll mul(ll a, ll b) { return (a * b) % MOD; }
ll power(ll base, ll exp) {
    ll res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp % 2 == 1) res = mul(res, base);
        base = mul(base, base);
        exp /= 2;
    }
    return res;
}
ll modInverse(ll n) {
    return power(n, MOD - 2); // Valid only if MOD is prime
}
// Finds the shortest linear recurrence transition array for a given sequence S
vector<ll> BerlekampMassey(const vector<ll>& S) {
    int n = S.size(), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;
    ll b = 1;

    for (int i = 0; i < n; i++) {
        ++m;
        ll d = S[i] % MOD;
        for (int j = 1; j <= L; j++) d = add(d, mul(C[j], S[i - j]));
        if (d == 0) continue;

        T = C;
        ll coef = mul(d, modInverse(b));
        for (int j = m; j < n; j++) C[j] = sub(C[j], mul(coef, B[j - m]));

        if (2 * L > i) continue;
        L = i + 1 - L;
        B = T;
        b = d;
        m = 0;
    }
    C.resize(L + 1);
    C.erase(C.begin());
    for (auto &x : C) x = sub(0, x);
    return C;
}
// Multiplies two polynomials modulo the characteristic polynomial (tr)
vector<ll> combine(int n, const vector<ll>& a, const vector<ll>& b, const vector<ll>& tr) {
    vector<ll> res(n * 2 + 1, 0);
    for (int i = 0; i < n + 1; i++) {
        for (int j = 0; j < n + 1; j++) {
```

```
            res[i + j] = add(res[i + j], mul(a[i], b[j]));
        }
    }
    for (int i = 2 * n; i > n; --i) {
        for (int j = 0; j < n; j++) {
            res[i - 1 - j] = add(res[i - 1 - j], mul(res[i], tr[j]));
        }
    }
    res.resize(n + 1);
    return res;
}
// S contains the initial sequence values, 'tr' is the transition array from BM.
// K is 0-indexed, S and tr must be same size
ll LinearRecurrence(const vector<ll>& S, const vector<ll>& tr, ll k) { // O(N^2 log K)
    assert(S.size() == tr.size());
    int n = tr.size();
    if (n == 0) return 0;
    if (k < n) return S[k] % MOD;

    vector<ll> pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(n, pol, e, tr);
        e = combine(n, e, e, tr);
    }

    ll res = 0;
    for (int i = 0; i < n; i++)
        res = add(res, mul(pol[i + 1], S[i]));
    return res;
}
```

Product of divisors

```
long long productOfDivisors(const vector<pair<long long, long long> > &primeFactors) {
    // We compute the exponent modulo 2 * (MOD - 1) to safely divide by 2 later
    long long expMod = 2LL * (MOD - 1);
    // 1. Calculate the total number of divisors d(N) modulo 2*(MOD - 1)
    long long d_N = 1;
    for (const auto &factor: primeFactors) {
        long long a_i = factor.second;
        d_N = (d_N * (a_i + 1)) % expMod;
    }
    long long product = 1;
    // 2. Calculate the contribution of each prime factor
    for (const auto &factor: primeFactors) {
        long long p_i = factor.first;
        long long a_i = factor.second;
        // Exponent for this prime factor: (a_i * d_N)
        long long exponent = (a_i * d_N) % expMod;
        // Safe division by 2 because (a_i * d_N) is mathematically guaranteed to be even
        exponent /= 2;
        // Multiply the contribution to the final product
        product = (product * power(p_i, exponent)) % MOD;
```

```
    }
    return product;
}
```